Docker: где живут данные

Слой контейнера, volumes и bind mount / 12-14 октября 2026

Моя задача и итог

Я продолжил Docker после Session 13: проверил, где хранятся изменения и как отделить данные от контейнера. Практику выполнял под linux01 на Ubuntu VM, с подсказками. Dockerfile уже выдан как следующий блок, но сборку своего образа я ещё не подтвердил.

Три разных места хранения

- Изменяемый слой принадлежит конкретному контейнеру. Stop/start его не удаляет; удаление контейнера удаляет этот слой. Новый контейнер из прежнего образа не получает изменения старого.
- Именованный volume управляется Docker и существует независимо от подключённого контейнера. Удаление временного контейнера с --rm не удаляет наш именованный volume.
- Bind mount подключает конкретную папку хоста. В моём случае хост - Ubuntu, не Mac. Файл на Ubuntu доступен контейнеру через выбранный путь, а не копируется туда.

Изменяемый слой: мой эксперимент

12 октября я показал s18-web с прежним ID 0e4c538a2d84 в состоянии Exited (0). Результат проверки Mounts=[] сообщил в чате. Затем запустил контейнер и создал /tmp/layer-test.txt через exec. Из-за /n вместо \n в файл попал буквальный текст container-layer-test/n; для проверки хранения это не мешало.

Я сообщил, что файл сохранился после stop/start. Скриншот docker diff показал A /tmp/layer-test.txt, а также другие изменения Nginx. В новом временном контейнере из nginx:1.30.5-alpine проверка показала File is absent. Образ не получил файл старого контейнера.

Новые приёмы наблюдения

```
sudo docker inspect --format '{{json .Mounts}}' s18-web
sudo docker diff s18-web
sudo docker diff s18-web | grep -Fx 'A /tmp/layer-test.txt'
```

Mounts показывает подключения; пустой список не означает отсутствие файлов. Diff показывает изменения в файловой системе контейнера: A - добавлено, C - изменено, D - удалено. Это не журнал содержимого volume.

Фильтрацию grep разобрал как приём: ^A отбирает добавления по началу строки, -F ищет буквальный текст, -x требует совпадения всей строки. Отдельный запуск фильтра на скриншоте не показан.

Старые run, start, stop, ps, curl и коды завершения повторно не разбираю: они описаны в предыдущих рефератах Docker и Bash.

Именованный volume

Один контейнер записал, другой прочитал

Что я сделал

Создание training-volume-01 сообщил в чате. Первый временный контейнер подключил его к /data и записал probe.txt с текстом volume-test. После исправления команды на скриншоте показан код 0. Контейнер запускался с --rm; отдельный список после его удаления я не показывал.

Другой контейнер подключил тот же volume к /date и прочитал /date/probe.txt: скриншот показывает volume-test. Разные пути подключения не мешают обращаться к одним данным, если путь команды совпадает с точкой подключения.

Новая модель параметров

```
--mount type=volume,source=training-volume-01,target=/data
-v training-volume-01:/date:ro
```

Первый вариант явно называет тип, источник и путь внутри контейнера. Второй короче: имя volume : путь внутри контейнера : режим. В этой практике я попросил короткие команды; дополнительные ограничения разбираем отдельно.

Ro запрещает запись через это подключение. Другой контейнер может подключить тот же volume для записи. Это не глобальная блокировка хранилища.

```
sudo docker volume ls --filter name=training-volume-01
sudo docker volume create training-volume-01
```

Это справочник новых команд, не указание повторно создавать volume. Перед созданием мы проверяем точное имя; фильтр может показывать похожие имена.

Ошибки, которые я разобрал

- При переносе многострочной команды в одну строку оставил \ перед пробелами: Docker получил invalid reference format. Перенос команды работает только с \ непосредственно перед переводом строки. В однострочном if нужны разделители перед else и fi.
- Пропустил --network перед none: Docker принял none за имя образа и попытался получить none:latest. Docker login не требовался; исправили аргументы.
- Подключил volume к /date, но читал из /data: получил No such file or directory. Изменил путь чтения и успешно прочитал файл.
- При проверке ro сначала снова обратился к /data вместо /date. Код 1 был получен по неверной причине. После исправления сообщил об ошибке Read-only file system.

Что это доказало

Показанное чтение из другого контейнера подтверждает независимость данных от первого. Отказ записи через ro подтверждён моим сообщением, не отдельным скриншотом финального отказа. Один лишь ненулевой код не доказывает нужную причину ошибки.

Bind mount и моя веб-страница

Изменение с хоста и граница readonly

Схема практики

Я создал /home/linux01/bind-site/index.html на Ubuntu и подключил папку к /usr/share/nginx/html в контейнере bind-web. Учебный адрес - http://127.0.0.1:8134/. Подключение только для чтения:

```
-v /home/linux01/bind-site:/usr/share/nginx/html:ro
```

В отличие от именованного volume источником здесь служит абсолютный путь хоста. Подключение закрывает исходное содержимое каталога назначения, не изменяя сам базовый образ.

Результаты по моему итоговому сообщению

- Первая версия страницы ответила HTTP 200.
- После изменения файла на Ubuntu появилась вторая версия без перезапуска контейнера.
- В Mounts указаны Type=bind и RW=false.
- После повторной проверки я сообщил об ожидаемом запрете записи. Причина первоначально сообщённого кода 0 не установлена: вывод не был предоставлен.
- Я отдельно подтвердил, что bind-web остановлен. Удаление контейнера и папки не выполнялось в подтверждённой практике.

Эти результаты сообщены мной в чате; новых скриншотов bind mount и независимого аудита VM нет.

Что я исправил в понимании

Сначала я спутал bind mount с volume и предположил, что файл исчезнет вместе с контейнером. После объяснения уточнил: файл остаётся в папке Ubuntu. Ro относится к пути подключения контейнера, а пользователь хоста сохраняет свои права. Изменения на хосте имеют последствия - их видит сайт.

На контрольный вопрос ответил правильно: новый контейнер с этой же подключённой папкой увидит v2. Реальное удаление и пересоздание bind-web ради этой проверки не выполнялось; это проверка понимания, не отдельный эксперимент.

Состояние и следующий шаг

Training-volume-01 и исходные файлы сохраняются; их удаление не подтверждено. Текущее состояние s18-web после перезагрузки не проверено. 13 октября снова был SSH timeout; я сообщил, что после перезагрузки сервера доступ восстановился. Первопричина не установлена.

Блок хранения зачтён с подсказками, числовую самостоятельность не повышаю. Следующий выданный блок - Dockerfile с FROM/COPY, образы v1/v2; результатов сборки пока нет. Compose остаётся впереди.

Материалы веду также в Obsidian, PDF и Git сохраняю. Фоновой синхронизации нет. Учусь цельными блоками с общей проверкой результата.

Справочник: https://docs.docker.com/engine/storage/bind-mounts/